Rapport de TP n°3

Implémentation d'un Système Expert d'ordre 0^+

IA01 - A25

Université de Technologie de Compiègne

Youssef KHEMIRA

28 mars 2026

Table des matières

1 Rappel et préparation **2**
1.1 Domaine et objectif du Système Expert 2
1.2 Logique générale des règles 2

2 Implémentation du moteur d'inférence **3**
2.1 Structure de la base de connaissances 3
2.1.1 Les faits 3
2.2 Les règles 4
2.3 Chaînage avant 5
2.4 Chaînage arrière 6

3 Interaction et interface **7**
3.1 Hunchentoot et helpers HTTP 7
3.2 Interface Javascript 9

4 Autres informations **13**
4.1 Base de données SQLite 13
4.2 Image Docker et compilation Common Lisp 13
4.2.1 Structure de l'image Docker 13
4.2.2 Compilation et exécution du code Lisp 13
4.2.3 Hébergement sur Render 14

5 Usage de l'IA générative **14**
5.1 Extension de la base de connaissances 14
5.2 Transcription en SQL 14
5.3 Recherche de stack technique Lisp 15
5.4 Implémentation de l'endpoint de diagnostic 15

6 Discussion et conclusion **15**

Introduction

Ce TP consiste à réaliser le système expert proposé au TP 2, capable diagnostiquer des pannes informatiques et de proposer des actions à partir de symptômes fournis par l'utilisateur. Il s'appuie sur une base de connaissances initiale et un moteur d'inférence (chaînage avant et arrière).

1 Rappel et préparation

1.1 Domaine et objectif du Système Expert

Pour rappel, le domaine d'expertise choisi est celui de la maintenance informatique, la problématique est en particulier le diagnostic des pannes, pour un service après vente par exemple, ou un guide d'auto résolution d'un problème. C'est une problématique qui peut difficilement être résolue par des algorithmes déterministes ou mathématiques car la complexité devient très importante au vu du nombre de causes et de conséquences possibles. En effet, une même panne peut avoir des origines différentes, tandis d'une même origine peut causer des pannes différentes. De plus, le diagnostic est souvent visuel, approximatif et subjectif, il est donc difficile résoudre le problème avec des calculs numériques.

L'objectif de ce Système Expert est de déterminer la nature de la panne qu'elle soit matérielle ou logicielle. Il indiquera à l'utilisateur le ou les composants ou logiciels responsables de cette panne ainsi que le niveau de gravité. Ensuite, il peut recommander des actions de réparation à partir des symptômes identifiés, comme réinstaller des pilotes, redémarrer en mode sans échec, vérifier les branchements, installer ou supprimer un logiciel ; ou encore des actions de diagnostic qui vont affiner sa résolution, comme une vérification de la RAM ou du disque dur.

L'ordre 0^+ est adapté à ce système car les faits peuvent êtres hiérarchisés et on peut imbriquer les règles avec une profondeur supérieure à 2 pour avoir plusieurs conditions. Les valeurs peuvent être associées à des attributs. Le niveau d'abstraction est plus élevé que l'ordre 0, qui aurait été trop limité vu le nombre de cas possibles à traiter, et l'ordre 1 est trop complexe inutilement pour cette tâche, l'ordre 0^+ est donc le bon compromis.

1.2 Logique générale des règles

Afin de simuler l'expertise d'un spécialiste dans un domaine, le système expert a besoin de plusieurs informations. Ces informations sont contenues dans deux bases différentes : la base de faits et la base de règles. La base de faits contient les informations dont on dispose avant l'expertise, ce sont les informations initiales, aussi appelée mémoire court terme. La base de règles contient l'ensemble des règles qui peuvent être déduites des informations initiales après l'expertise, c'est la mémoire long terme. L'expertise est réalisée par un moteur d'inférence, qui va confronter les faits aux règles.

Dans notre cas, les règles sont des implications conditionnelles, où un ensemble de symptômes et d'observations nous permettent d'établir un diagnostic de la panne.

Nous avons fait le choix d'un système d'ordre 0^+ car dans ce type de système, un fait est une valeur, par exemple $(temps = pluie) \rightarrow (transport = voiture)$. L'ordre 0^+ est adapté à ce système car les faits peuvent êtres hiérarchisés et on peut imbriquer les règles avec une profondeur supérieure à 2 pour avoir plusieurs conditions. Les valeurs peuvent être associées à des attributs. Le niveau d'abstraction est plus élevé que l'ordre

0, qui aurait été trop limité vu le nombre de cas possibles à traiter, et l'ordre 1 est trop complexe inutilement pour cette tâche, l'ordre 0^+ est donc le bon compromis.

Le moteur d'inférence par chaînage avant est pertinent car il permet de retrouver la cause probable liée à plusieurs symptômes. Les règles sont indépendantes

Dans notre travail, nous avons veillé à ce que les règles restent lisibles et modulaires, afin de faciliter leur maintenance et leur extension. Chaque règle est indépendante, mais l'ensemble forme un réseau cohérent qui permet de couvrir les principaux scénarios de pannes étudiés. Par exemple, une règle formulée sur la détection d'un bruit anormal du disque dur conduit directement à une hypothèse de défaillance mécanique, tandis qu'une autre, fondée sur l'absence de démarrage du système, oriente vers un problème d'alimentation ou de carte mère. Ces exemples illustrent la manière dont la logique conditionnelle se traduit en diagnostics concrets, en cohérence avec le domaine d'expertise choisi.

2 Implémentation du moteur d'inférence

2.1 Structure de la base de connaissances

2.1.1 Les faits

Les faits représentent les symptômes ou observations que l'utilisateur peut constater sur son ordinateur. Ils sont stockés dans une base de données SQLite dans une table facts ayant trois colonnes : un code unique (ex : F1), un texte en langage naturel compréhensible par l'utilisateur qui sera affiché (ex : "Mon ordinateur est devenu très lent"), et une catégorie permettant de regrouper les symptômes par thème pour l'interface utilisateur (Performance, Démarrage, Réseau, Thermique, Sécurité, Disque, Matériel, Système, Logiciels, Périphériques, Audio, Affichage).

Le code Lisp charge ces faits depuis la base de données via la fonction load-facts, qui exécute une requête SQL et transforme chaque ligne en une structure de données Lisp sous forme de liste de propriétés (property list) contenant le code, le label et la catégorie.

```
(defun load-facts ()
  (mapcar
   (lambda (l)
     (destructuring-bind (code label category) (uiop:split-string
        l :separator "|")
       (list :code code :label label :category category)))
   (sql "SELECT code||'|'||label||'|'||category FROM facts;")))
```

La fonction load-facts commence par appeler la fonction sql qui exécute une requête SQLite et retourne les résultats sous forme de liste de chaînes de caractères. Les 3 colonnes obtenues (de la table facts) sont concaténées sur une seule ligne par le caractère "|".

La fonction mapcar applique ensuite une fonction lambda à chaque élément de la liste retournée par la requête SQL. Pour chaque ligne, la fonction uiop :split-string découpe la chaîne en utilisant le même séparateur "|". La macro destructuring-bind assigne directement les trois parties obtenues aux variables code, label et category. Finalement, la fonction list construit une liste de propriétés avec les mots-clés :code, :label et :category, afin de pouvoir structurer correctement les données et les récupérer plus facilement.

2.2 Les règles

Les règles sont stockées dans une table rules comportant quatre colonnes : un identifiant unique, les conditions séparées par des points-virgules (par exemple F1 ;F7), la conclusion de la règle, et une description de l'action effectuée par l'utilisateur (utilisée pour le chaînage arrière).

Chaque règle suit le format SI conditions ALORS conclusion. Les conditions sont des codes de faits ou des conclusions intermédiaires, et les conclusions peuvent être de deux types : soit une cause possible (par exemple CAUSE_POSSIBLE(pc_lent,surcharge _logicielle_processus_gourmands)), soit une action recommandée (par exemple ACTION _RECOMMANDEE(pc_lent,PRECIS :Vos symptômes indiquent...)).

Le système utilise un mécanisme de chaînage en deux niveaux : des règles de premier niveau déduisent des causes possibles à partir des symptômes, puis des règles de second niveau déduisent des actions recommandées à partir de ces causes possibles. Par exemple, la règle R1 stipule que si l'utilisateur observe les symptômes F1 (ordinateur lent) et F7 (processeur utilisé à plus de 80 %), alors on peut déduire une cause possible de surcharge logicielle. Ensuite, la règle R2 stipule que si cette cause possible est présente, alors on recommande une action précise pour résoudre le problème. La plupart des règles de premier niveau disposent d'une action recommandée, cette action est présentée à l'utilisateur comme conseil général car il n'est fondé que sur 1 symptôme. Cependant, dans le cas où plusieurs causes possibles convergent vers une même action, cette dernière est indiquée comme fortement probable.

La fonction load-rules charge les règles et les transforme en structures de données Lisp exploitables par les moteurs d'inférence :

```
(defun load-rules ()
  (mapcar
   (lambda (l)
     (destructuring-bind (id conds then action-done)
        (uiop:split-string l :separator "|")
       (list :id id
             :conds (uiop:split-string conds :separator ";")
             :then then
             :action-done (if (string= action-done "") nil
                action-done))))
   (sql "SELECT
      id||'|'||conds||'|'||conclusion||'|'||COALESCE(action_done,'')
      FROM rules;")))
```

La fonction load-rules est similaire à load-facts. Elle exécute une requête SQL qui concatène les quatre colonnes de la table rules avec le séparateur "|". La fonction SQL COALESCE remplace les valeurs NULL par une chaîne vide pour éviter les erreurs de parsing côté Lisp.

Pour chaque ligne retournée, mapcar applique une fonction lambda qui découpe la chaîne avec uiop :split-string et assigne les quatre parties aux variables id, conds, then et action-done via destructuring-bind. Cette fois, il faut prendre un compte un paramètre important qui est que le champ conds contient potentiellement plusieurs conditions séparées par des points-virgules : un second appel à uiop :split-string avec le séparateur « ; » transforme cette chaîne en liste de conditions individuelles. La fonction string= compare deux chaînes de caractères et retourne vrai si elles sont identiques. On s'en sert pour transformer les chaînes vides en nil afin de faciliter les tests ultérieurs.

2.3 Chaînage avant

Pour ce projet, il a été décidé d'implémenter deux types de chaînage : le chaînage avant et le chaînage arrière. Le chaînage avant correspond parfaitement à l'objectif de diagnostic de panne informatique, car il permet de partir des symptômes observés pour déduire les causes et recommander des actions. Le chaînage arrière, bien que moins utile dans ce contexte, apporte une fonctionnalité complémentaire : il permet de visualiser quels problèmes pourraient être évités ou résolus en effectuant une action préventive donnée.

Les deux moteurs d'inférence sont implémentés en Common Lisp.

```
(defun forward-chain (facts)
  (let ((known (copy-list facts)) (trace '()) (rules (load-rules))
     (changed t))
    (loop while changed do
      (setf changed nil)
      (dolist (r rules)
        (when (and (every (lambda (c) (member c known :test
           #'string=)) (getf r :conds))
                   (not (member (getf r :then) known :test
                      #'string=)))
          (push (getf r :then) known)
          (push (format nil "~A déclenché -> ~A" (getf r :id)
             (getf r :then)) trace)
          (setf changed t))))
    (list :facts known :trace (nreverse trace) :actions
       (remove-if-not (lambda (f) (search "ACTION_RECOMMANDEE" f))
       known))))
```

La fonction forward-chain prend en paramètre facts, la liste des codes de faits correspondant aux symptômes sélectionnés par l'utilisateur (par exemple ("F1" "F7")).

On commence par créer un bloc avec des variables locales. Quatre variables sont initialisées : known reçoit une copie de la liste des faits initiaux via copy-list (fonction native qui crée une copie superficielle d'une liste pour éviter de modifier la liste originale) ; trace est initialisée à une liste vide '() qui contiendra l'historique des règles déclenchées ; rules reçoit le résultat de load-rules (toutes les règles du système) ; changed est initialisé à t (équivalent de True) pour permettre l'entrée dans la boucle.

On entre ensuite dans une boucle qui continue tant que changed est vrai. Au début de chaque itération, setf remet changed à nil. La macro dolist parcourt ensuite chaque règle r de la liste rules.

Pour chaque règle, la forme spéciale when (équivalent de if sans else) vérifie une condition composée de deux parties reliées par and. La première partie utilise every, qui retourne vrai si son prédicat (ici une fonction lambda) est satisfait pour tous les éléments de la liste. Cette lambda vérifie pour chaque condition c de la règle si elle est présente dans les faits connus via member, une fonction qui cherche un élément dans une liste. L'option :test #'string= indique d'utiliser la comparaison de chaînes à la place de la comparaison par défaut. La fonction getf extrait la valeur associée à un mot-clé (:conds) dans notre property list. La seconde partie de la condition vérifie avec not et member que la conclusion de la règle n'est pas déjà dans les faits connus.

Si les deux conditions sont satisfaites, push ajoute la conclusion aux faits connus et ajoute une entrée formatée à la trace. La fonction format fonctionne comme printf en C : nil comme premier argument indique de retourner une chaîne plutôt que d'afficher, et A

est un placeholder pour insérer une valeur. Enfin, changed est remis à t pour signaler qu'une modification a eu lieu.

Après la boucle, la fonction retourne une liste de trois propriétés. La propriété :trace utilise nreverse, une fonction qui inverse une liste, nécessaire car push ajoute les éléments devant donc l'ordre est inversé. La propriété :actions utilise remove-if-not, qui filtre une liste en ne gardant que les éléments satisfaisant un prédicat. Ce prédicat utilise search, qui cherche une sous-chaîne dans une chaîne et retourne sa position (ou nil si non trouvée).

Le chaînage avant fonctionne en saturation : il parcourt répétitivement toutes les règles jusqu'à ce qu'aucune nouvelle déduction ne soit possible.

2.4 Chaînage arrière

Le chaînage arrière adopte une approche inverse : en partant d'une action de maintenance, il remonte la chaîne des règles pour identifier quels symptômes ou problèmes initiaux son été potentiellement évités grâce à cette action.

```
(defun backward-chain (selected-actions)
  (let ((rules (load-rules))
        (resolved-problems '())
        (trace '()))
    (dolist (action selected-actions)
      (dolist (r rules)
        (when (string= (getf r :then) action)
          (dolist (cond (getf r :conds))
            (dolist (r2 rules)
              (when (string= (getf r2 :then) cond)
                (dolist (fact-code (getf r2 :conds))
                  (unless (search "CAUSE_POSSIBLE" fact-code)
                    (push fact-code resolved-problems)
                    (push (format nil "~A peut résoudre le
                       problème ~A" action fact-code)
                       trace))))))))
    (list :resolved (remove-duplicates resolved-problems :test
       #'string=)
          :trace (nreverse trace)))\
```

La fonction backward-chain prend en paramètre selected-actions, la liste des codes d'actions sélectionnées par l'utilisateur (par exemple ("ACTION_RECOMMANDEE(pc _lent,PRECIS :...)")).

Comme pour le chaînage avant, let initialise trois variables locales : rules contient toutes les règles, resolved-problems est une liste vide qui contiendra les codes des problèmes pouvant être résolus, et trace enregistrera le raisonnement.

L'algorithme utilise quatre boucles dolist imbriquées. La première parcourt chaque action sélectionnée par l'utilisateur. La deuxième parcourt toutes les règles pour trouver celles dont la conclusion (:then) correspond exactement à l'action recherchée, via string=. Lorsqu'une telle règle est trouvée, la troisième boucle parcourt ses conditions (:conds), qui sont typiquement des CAUSE_POSSIBLE. La quatrième boucle cherche ensuite les règles dont la conclusion est cette cause possible, remontant ainsi d'un niveau dans la chaîne de raisonnement.

Pour chaque condition de ces règles de niveau supérieur, la forme spéciale unless (l'inverse de when, exécute le code si la condition est fausse) vérifie que le code ne contient

pas "CAUSE_POSSIBLE". Cela permet de ne garder que les faits de base (les symptômes) et d'exclure les conclusions intermédiaires. Si c'est un fait de base, il est ajouté à resolved-problems et une explication est ajoutée à la trace.

À la fin, la fonction retourne les problèmes résolus en utilisant remove-duplicates, une fonction qui élimine les doublons d'une liste. L'option :test #'string= indique d'utiliser la comparaison de chaînes pour déterminer si deux éléments sont identiques.

Cette fonctionnalité permet à l'utilisateur de répondre à la question : "J'ai fait ça sur mon ordinateur, qu'est ce que ça change ?"

3 Interaction et interface

Pour ce système expert, le choix s'est porté sur une interface web plutôt qu'une interface en ligne de commande ou une application native. Ce choix se justifie par plusieurs avantages. Premièrement, l'accessibilité : une interface web est accessible depuis n'importe quel navigateur sans installation préalable, ce qui facilite le déploiement et l'utilisation. Deuxièmement, l'ergonomie : une interface graphique permet de présenter les symptômes de manière claire et organisée, avec des cases à cocher intuitives à la place des commandes textuelles. Troisièmement, la séparation des responsabilités : le moteur d'inférence en Common Lisp gère la logique métier tandis que l'interface web gère la présentation, ce qui rend le code plus maintenable.

L'architecture repose sur un modèle client-serveur classique : le serveur Common Lisp expose une API REST que le client JavaScript consomme pour afficher les données et soumettre les requêtes de diagnostic.

3.1 Hunchentoot et helpers HTTP

Bibliothèques Hunchentoot est un serveur web écrit entièrement en Common Lisp. Il est chargé au démarrage de l'application via Quicklisp, le gestionnaire de paquets de Common Lisp :

```
(ql:quickload '(:hunchentoot :cl-json :uiop))
```

Cette ligne charge trois bibliothèques : Hunchentoot pour le serveur HTTP, CL-JSON pour la sérialisation et désérialisation JSON, et UIOP pour les utilitaires système (exécution de programmes externes, variables d'environnement).

Démarrage du serveur Le serveur est démarré à la fin du fichier principal :

```
(start (make-instance 'easy-acceptor :port (port)))
(loop (sleep 3600))
```

La fonction make-instance native à Common Lisp crée une instance de la classe easy-acceptor fournie par Hunchentoot. Cette classe représente un serveur HTTP simple qui écoute sur un port donné. La fonction port lit la variable d'environnement PORT ou utilise la valeur par défaut 10000. La boucle (loop (sleep 3600)) maintient le programme en vie indéfiniment, car sans elle le programme se terminerait immédiatement après le démarrage du serveur.

Fichiers statiques L'interface web est composée de fichiers statiques (HTML, CSS, JavaScript) qui doivent être servis par le serveur :

```
(push (create-folder-dispatcher-and-handler "/static/"
   #p"/app/static/") *dispatch-table*)
(push (create-static-file-dispatcher-and-handler "/"
   #p"/app/static/index.html") *dispatch-table*)
```

La variable *dispatch-table* est une variable globale de Hunchentoot qui contient la liste des règles de routage. La fonction push ajoute un élément en tête de liste. La fonction create-folder-dispatcher-and-handler crée un gestionnaire qui sert tous les fichiers d'un répertoire : toute requête commençant par static sera mappée vers le répertoire /app/static/. La notation #p est une syntaxe Common Lisp pour créer un objet pathname (chemin de fichier). La fonction create-static-file-dispatcher-and-handler crée un gestionnaire pour servir un fichier spécifique : la racine / sert directement le fichier index.html.

Endpoints API Les endpoints de l'API sont définis avec la macro define-easy-handler de Hunchentoot :

```
(define-easy-handler (api-facts :uri "/api/facts") ()
  (let ((facts (load-facts))
        (result-array '()))
    (dolist (f facts)
      (push (list (getf f :code) (getf f :label) (getf f
         :category)) result-array))
    (let ((json-obj (make-hash-table :test 'equal)))
      (setf (gethash "facts" json-obj) (nreverse result-array))
      (json json-obj))))
```

La macro define-easy-handler prend en premier argument une liste contenant le nom du handler et l'URI associée. Le second argument (ici vide ()) permet de déclarer des paramètres GET ou POST. Le corps de la fonction est exécuté à chaque requête sur cette URI.

Ce handler charge tous les faits depuis la base de données, les transforme en une liste de listes (format plus compact pour JSON), puis crée une table de hachage avec make-hash-table. L'option :test 'equal indique d'utiliser la comparaison structurelle pour les clés (nécessaire pour les chaînes de caractères). La fonction gethash accède à une entrée de la table, et combinée avec setf elle permet d'assigner une valeur. Enfin, la fonction json sérialise l'objet en JSON.

Fonction helper JSON

```
(defun json (obj)
  (setf (content-type*) "application/json")
  (with-output-to-string (s)
    (cl-json:encode-json obj s)))
```

Cette fonction utilitaire prépare une réponse JSON. La fonction content-type* est fournie par Hunchentoot et permet de définir l'en-tête Content-Type de la réponse HTTP. La macro with-output-to-string crée un flux de sortie vers une chaîne de caractères : tout ce qui est écrit sur ce flux est capturé dans une chaîne retournée à la fin. La fonction

cl-json :encode-json, de la bibliothèque CL-JSON, sérialise un objet Lisp (liste, table de hachage, etc.) en JSON.

Endpoint principal de diagnostic

```
(define-easy-handler (api-run :uri "/api/diagnostic") ()
  (let* ((raw-json (raw-post-data :force-text t))
         (p (cl-json:decode-json-from-string raw-json))
         (facts (cdr (assoc :facts p)))
         (mode (cdr (assoc :mode p)))
         (selected-actions (cdr (assoc :selected-actions p)))
         (json-obj (make-hash-table :test 'equal)))
    (if (string= mode "backward")
        ;; Chaînage arrière
        (let* ((result (backward-chain selected-actions))
               (resolved-codes (getf result :resolved))
               (resolved-facts (get-fact-labels resolved-codes)))
          (setf (gethash "mode" json-obj) "backward")
          (setf (gethash "resolved_problems" json-obj)
                (mapcar (lambda (f)
                          (list (getf f :code) (getf f :label)
                                (getf f :category)))
                        resolved-facts))
          (setf (gethash "trace" json-obj) (getf result :trace)))
        ;; Chaînage avant
        (let ((result (forward-chain facts)))
          (setf (gethash "mode" json-obj) "forward")
          (setf (gethash "inference_result" json-obj) (getf result
             :trace))
          (setf (gethash "rule_conclusions" json-obj) (getf result
             :actions))))
    (json json-obj)))
```

C'est l'endpoint principal de l'API. Contrairement au let, la forme spéciale let* évalue les liaisons séquentiellement, ce qui permet à une variable d'utiliser la valeur d'une variable précédente. La fonction raw-post-data de Hunchentoot récupère le corps brut de la requête POST ; l'option :force-text t force l'interprétation comme texte. La fonction cl-json :decode-json-from-string désérialise le JSON en une liste associative Lisp (alist). La fonction assoc cherche ensuite une paire clé-valeur dans une alist, et cdr extrait la valeur (le second élément de la paire).

Selon le mode spécifié dans la requête, l'endpoint appelle soit forward-chain soit backward-chain, puis formate le résultat en JSON pour le renvoyer au client.

3.2 Interface Javascript

Le fichier HTML définit la structure de l'interface avec plusieurs écrans :

```
<div id="welcomeScreen" class="screen active fade-in">
    <!-- Écran d'accueil avec choix du mode -->
</div>

<div id="wizardScreen" class="screen fade-in">
```

```
    <!-- Assistant pas-à-pas pour sélectionner les symptômes ou actions -->
</div>

<div id="resultsScreen" class="screen fade-in">
    <!-- Affichage des résultats du diagnostic -->
</div>

<div id="loadingScreen" class="screen fade-in">
    <!-- Écran de chargement pendant l'analyse -->
</div>
```

L'interface utilise un système d'écrans où un seul écran est visible à la fois, contrôlé par la classe CSS "active". L'écran d'accueil propose deux boutons pour choisir entre le diagnostic (chaînage avant) et l'analyse d'impact (chaînage arrière). En choisissant un des boutons, on arrive sur l'écran wizard qui guide l'utilisateur à travers les catégories de symptômes ou d'actions selon le chaînage. L'écran de résultats affiche les conclusions du moteur d'inférence.

Variables globales Javascript

```
let allFacts = [];
let allActions = [];
let currentMode = null;
let selectedFacts = [];
let selectedActions = [];
let currentStep = 0;
let categories = [];
let actionCategories = [];
let grouped = {};
let groupedActions = {};
```

Le Javascript stocke l'état de l'application dans des variables globales.

- allFacts et allActions stockent les données chargées depuis l'API ;
- currentMode indique si l'utilisateur est en mode chaînage avant ("forward") ou arrière ("backward") ;
- selectedFacts et selectedActions contiennent les sélections de l'utilisateur ;
- currentStep suit la progression dans l'assistant ;
- grouped et groupedActions organisent les données par catégorie pour l'affichage.

Chargement des données depuis l'API

```
async function loadFacts() {
  try {
    const response = await fetch("/api/facts");
    if (!response.ok) {
      throw new Error('HTTP error status: ${response.status}');
    }
    const data = await response.json();
    allFacts = data.facts || [];
```

```

    grouped = {};
    allFacts.forEach(fact => {
      const category = fact[2] || 'Autre';
      if (!grouped[category]) {
        grouped[category] = [];
      }
      grouped[category].push(fact);
    });

    categories = Object.keys(grouped).sort();
    renderAllCategories();
  } catch (error) {
    console.error("Erreur:", error);
  }
}
```

Cette fonction utilise l'API Fetch que l'on retrouve dans tous les navigateurs modernes, pour effectuer une requête HTTP GET vers '/api/facts'. Le mot-clé async permet d'utiliser await pour attendre la réponse de manière asynchrone sans bloquer l'interface. La méthode response.json() parse automatiquement la réponse JSON. Une fois les faits chargés, ils sont regroupés par catégorie dans l'objet grouped pour faciliter l'affichage par étapes. La fonction renderAllCategories génère ensuite le HTML correspondant.

```
async function runDiagnostic() {
  showScreen("loadingScreen");
  try {
    let payload;
    if (currentMode === "forward") {
      updateSelectedFacts();
      payload = { facts: selectedFacts, mode: "forward", goal: ""
         };
    } else {
      updateSelectedActions();
      payload = { facts: [], mode: "backward", goal: "",
         "selected-actions": selectedActions };
    }

    const response = await fetch("/api/diagnostic", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload)
    });

    if (!response.ok) throw new Error(`Status:
       ${response.status}`);
    const result = await response.json();
    displayResults(result);
  } catch (error) {
    console.error("Erreur :", error);
  }
}
```

Cette fonction prépare et envoie la requête de diagnostic. Elle commence par afficher l'écran de chargement pour informer l'utilisateur que le traitement est en cours. Selon le mode, elle construit un objet payload contenant soit les faits sélectionnés (chaînage avant) soit les actions sélectionnées (chaînage arrière). La requête est envoyée en POST avec le header "Content-Type : application/json" et le corps sérialisé avec JSON.stringify. La réponse est ensuite parsée et passée à displayResults pour l'affichage.

```
function displayResults(result) {
  showScreen("resultsScreen");
  const actionsList = document.getElementById("actionsList");
  actionsList.innerHTML = "";

  if (currentMode === "backward" && result.resolved_problems) {
    displayBackwardResults(result.resolved_problems);
    return;
  }

  const preciseActions = [];
  const generalActions = [];

  if (result.rule_conclusions && result.rule_conclusions.length >
     0) {
    result.rule_conclusions.forEach(item => {
      const match =
         item.match(/ACTION_RECOMMANDEE\([^,]+,(.+)\)$/);
      if (match) {
        const message = match[1].trim();
        if (message.startsWith("PRECIS:")) {
          preciseActions.push(message.substring(7).trim());
        } else if (message.startsWith("GENERAL:")) {
          generalActions.push(message.substring(8).trim());
        } else {
          generalActions.push(message);
        }
      }
    });
  }
  // ... affichage des actions précises puis générales
}
```

La fonction d'affichage traite différemment les résultats selon le mode. En mode chaînage avant, elle parse les conclusions du moteur pour extraire les messages d'action. Une expression régulière extrait le contenu des 'ACTION_RECOMMANDEE'. Les actions sont ensuite triées en deux catégories : les diagnostics précis (préfixés par 'PRECIS :') qui correspondent à des combinaisons spécifiques de symptômes comme décrites plus haut dans le rapport, et les conseils généraux (préfixés par 'GENERAL :') qui sont des recommandations plus larges. Les diagnostics précis sont affichés en premier car ils sont plus pertinents.

Le reste du fichier Javascript contient d'autres fonctions de navigation et d'affichage, peu pertinentes à décrire dans le rapport.

La page HTML est stylisée avec une feuille CSS. L'interface utilise un design moderne

avec des dégradés de couleurs pour les boutons principaux, des ombres pour créer de la profondeur, et des transitions CSS pour les animations.

4 Autres informations

4.1 Base de données SQLite

Comme dit plus tôt, les faits et règles sont stockés dans une base de données SQLite. C'est un format de base de données qui a été choisi pour sa simplicité à mettre en place, son format en fichier qui ne nécessite pas d'accès avec des identifiants et IP, et sa portabilité. Le fait que ce soit une base de données permet toutefois d'avoir une meilleure structure pour stocker les informations, et surtout de potentiellement approvisionner la base de données en règles et faits à partir de sources extérieures de manière automatisée. Cela permet de séparer effectivement la logique des moteurs d'inférence avec la logique de stockage des informations.

Pour les besoins de démonstration de l'application, un fichier en Lisp initialisant la base de données a été créé, notamment pour la peupler à chaque initialisation de l'application, cette dernière étant réinitialisée à chaque utilisation dû au forfait gratuit de l'hébergement utilisé. Ce fichier Lisp utilise la librairie SQL et exécute simplement les requêtes définies au TP 2 (étendues).

4.2 Image Docker et compilation Common Lisp

4.2.1 Structure de l'image Docker

Le projet utilise une image Docker basée sur clfoundation/sbcl :latest, qui fournit un environnement d'exécution Steel Bank Common Lisp (SBCL). On construit l'image Docker en plusieurs étapes. D'abord, on installe les dépendances systèmes principales : sqlite3 pour la gestion de la base de données, et curl avec ca-certificates pour les téléchargements sécurisés. Ensuite, on installe Quicklisp qui est le gestionnaire de paquets de Common Lisp. Enfin, les bibliothèques dont on se sert dans le projet sont chargées pendant le build : Hunchentoot pour le serveur web, cl-json pour la sérialisation JSON, et uiop pour les opérations système.

4.2.2 Compilation et exécution du code Lisp

SBCL utilise une compilation JIT qui traduit le code Lisp en code machine natif lors du chargement des fichiers. Pendant la phase de build Docker, les dépendances (Hunchentoot, cl-json, uiop) sont précompilées via les commandes sbcl –non-interactive –eval '(ql :quickload ...)', ce qui génère des fichiers FASL (Fast Load) mis en cache dans le système Quicklisp. Au démarrage du conteneur, l'application est lancée avec sbcl –script /app/app.lisp, ce qui charge et compile le code source en mémoire, puis démarre le serveur web. Cette approche permet un démarrage rapide tout en bénéficiant des performances du code natif compilé par SBCL.

L'application elle-même est structurée en trois composants principaux copiés dans l'image : app.lisp contenant la logique métier et les endpoints HTTP, db-init.lisp pour l'initialisation de la base de données SQLite, et le dossier static contenant l'interface web (HTML, CSS, JavaScript). Un volume de données est créé dans /app/data pour persister

la base SQLite. Au démarrage du conteneur, la base de données est réinitialisée et le serveur Common Lisp est lancé en mode script.

4.2.3 Hébergement sur Render

L'application est déployée sur l'hébergeur Render. La plateforme détecte automatiquement le Dockerfile à la racine du projet et construit l'image lors de chaque déploiement. Le port d'écoute est configuré via la variable d'environnement PORT (par défaut 10000), permettant à Render d'acheminer le trafic HTTP vers le conteneur. Cette architecture bénéficie du déploiement continu depuis le dépôt Git.

5 Usage de l'IA générative

L'intelligence artificielle a fait basculer la manière de travailler dans de nombreux domaines technologiques. C'est pourquoi il est primordial, en tant que futurs ingénieurs, de s'habituer dès maintenant et au mieux aux pratiques d'usage de l'IA. C'est un outil qui ne disparaîtra pas et que nous serons amenés à utiliser tout au long de notre carrière.

En effet, tant que le fonctionnement est entièrement maîtrisé, que les résultats sont ceux escomptés et que l'architecture est parfaitement comprise, l'IA devient un outil exceptionnel et indispensable, permettant de gagner énormément de temps sur des tâches secondaires. L'aspect de supervision est alors renforcé, on peut donc davantage se concentrer sur l'efficacité, définir une structure claire aux applications réalisées, vérifier que tout fonctionne correctement.

Une estimation de la proportion de travail réalisée par l'IA est de 40%. Voici un détail de son utilisation au cours du projet.

5.1 Extension de la base de connaissances

L'IA générative a été utilisée pour renforcer la base de connaissances, en se basant sur ce qui a déjà été établi dans le TP 2. Un prompt donné est :

> [fichier texte contenant les faits et règles du TP 2]
> À partir du fichier fourni en pièce jointe, je voudrais que tu étendes la base de connaissances à plus d'une centaine de faits et règles. C'est un système expert sur la maintenance informatique. Les faits sont représentés de la manière suivante (3 colonnes) : code unique (à reprendre à partir du dernier code du fichier fourni), description en langage naturel, et catégorie (parmi Performance, Démarrage, Réseau, Thermique, Sécurité, Disque, Matériel, Système, Logiciels, Périphériques, Audio, Affichage). Les règles sont (4 colonnes) : identifiant unique, codes de conditions séparés par " ;" (F1 ;F2), conclusion (voir fichier) et description naturelle.

5.2 Transcription en SQL

Pour les besoins de l'application (stockage des connaissances dans une DB SQLite), il faut que la base de connaissances soit convertie en instructions SQL. Le prompt est :

> [fichier texte contenant les connaissances complètes initiales]
> Le backend de l'application tourne en Common Lisp. J'ai créé un fichier db-init en Common Lisp qui sera appelé à la fin du programme principal juste avant le démarrage du serveur HTTP. Ce fichier doit peupler la base de données SQLite contenant les connaissances grâce à des instructions SQL. À partir du fichier joint, crée les tables et les instructions SQL qui insèreront les faits et règles dans leurs tables respectives. Place le tout dans le fichier db-init de sorte à ce qu'il puisse directement être appelé depuis le programme principal.

5.3 Recherche de stack technique Lisp

Le Lisp n'étant pas un langage sur lequel on est habitué à travailler, les bibliothèques basiques sont peu connues. L'IA a donc été utilisée afin d'informer sur les possibilités qu'offre Lisp, notamment pour réaliser un serveur web et une API pour relier l'application.

> Je souhaite exposer mon moteur d'inférence Common Lisp via une API web simple. Quelles sont les bibliothèques standards pour créer un serveur HTTP (type Hunchentoot) et manipuler du JSON ? Donne-moi un exemple de configuration de base pour servir des fichiers statiques

5.4 Implémentation de l'endpoint de diagnostic

La réalisation d'endpoints est assez compliquée en Lisp. Il a donc été demandé à l'IA de fournir l'implémentation de l'endpoint principal, qui servira d'inspiration pour les autres endpoints.

> Écris une fonction define-easy-handler pour Hunchentoot qui : 1. Récupère un JSON en POST. 2. Décode le mode ('forward' ou 'backward'). 3. Appelle mes fonctions forward-chain ou backward-chain. 4. Retourne le résultat en JSON

L'IA a également été utilisée pour assister dans la création des commandes pour l'image Docker.

6 Discussion et conclusion

Le système expert développé remplit son objectif en montrant comment une base de connaissances et un moteur d'inférence peuvent guider la prise de décision. L'apport de l'IA a été majeur sur la génération de contenu (base de données) et le boilerplate (infrastructure web). Cependant, pour toute la partie organisationnelle et les fondations du projet, une supervision humaine rigoureuse a été nécessaire. En termes d'évolutions, l'intégration d'une application capable d'ajouter automatiquement de nouveaux symptômes et actions permettrait d'enrichir la base de connaissances en continu et de rendre le système plus adaptable. Cette perspective ouvre la voie à un outil évolutif mieux aligné avec les besoins réels des utilisateurs.

Pour rappel, le système expert tourne sur https://ia01-se.onrender.com/.